

GRAPH

PROBLEM 5

QUESTION:

DS	Session	Graph	Question Information	Level 1	Challenge 85
Question description You have an undirected graph consisting of n vertices with weighted edges. A simple cycle is a cycle of the graph without repeated vertices. Let the <i>weight</i> of the cycle be the XOR of weights of edges it consists of. Let's say the graph is <i>good</i> if all its <i>simple</i> cycles have weight 1. A graph is bad if it's not good. Initially, the graph is empty. Then q queries follow. Each query has the next type: $u\ v\ x$ — add edge between vertices u and v of weight x if it doesn't make the graph bad. For each query print, was the edge added or not. Constraints $3 \leq n \leq 3 \cdot 10^5$; $1 \leq q \leq 5 \cdot 10^5$ $1 \leq u, v \leq n$; $u \neq v$; $0 \leq x \leq 1$ Input The first line contains two integers n and q — the number of vertices and queries. Next q lines contain queries — one per line. Each query contains three integers u, v and x — the vertices of the edge and its weight. It's guaranteed that there are no multiple edges in the input. Output For each query, print YES if the edge was added to the graph, or NO otherwise (both case-insensitive).					

Logical Test Cases

Test Case 1 INPUT (STDIN) <pre>9 12 6 1 0 1 3 1 3 6 0 6 2 0 6 4 1 3 4 1 2 4 0 2 5 0 4 5 0 7 8 1 8 9 1 9 7 0</pre> EXPECTED OUTPUT <pre>YES YES YES YES YES NO YES YES NO YES YES YES NO</pre>	Test Case 2 INPUT (STDIN) <pre>9 6 1 3 1 3 6 0 6 4 1 2 4 0 4 5 0 7 8 1</pre> EXPECTED OUTPUT <pre>YES YES YES YES YES YES</pre>
--	--

Mandatory Test Cases

Test Case 1
KEYWORD
<pre>int dfs1(int np,int lst)</pre>

▼ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
12	1255	78

SOURCE CODE:

```
#include <stdio.h>

#define MAX_VERTS 300005

int dsu_parent[MAX_VERTS];
int path_parity[MAX_VERTS];

int trace_dsu(int v) {
    if (v == dsu_parent[v]) return v;
    int root = trace_dsu(dsu_parent[v]);
    path_parity[v] ^= path_parity[dsu_parent[v]];
    return dsu_parent[v] = root;
}

int main() {
    int vertices, query_count;
    if (scanf("%d %d", &vertices, &query_count) != 2) return 0;

    for (int i = 1; i <= vertices; i++) {
        dsu_parent[i] = i;
        path_parity[i] = 0;
    }

    while (query_count--) {
        int u, v, weight;
        scanf("%d %d %d", &u, &v, &weight);

        int ru = trace_dsu(u);
        int rv = trace_dsu(v);

        if (ru != rv) {
            dsu_parent[ru] = rv;
            path_parity[ru] = path_parity[u] ^ path_parity[v] ^ weight;
            printf("YES\n");
        } else {
            int cycle_weight = path_parity[u] ^ path_parity[v] ^ weight;
            if (cycle_weight == 1) {
                printf("YES\n");
            } else {
                printf("NO\n");
            }
        }
    }
    return 0;
}
```

OUTPUT:

```
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
YES
YES
YES
YES
YES
YES
```

```
9 6
1 3 1
3 6 0
6 4 1
2 4 0
4 5 0
7 8 1
YES
YES
YES
YES
YES
YES
```